

How are Iterable, Iterator and Generator Different?

The concepts — iterable, iterator and generator — can be pretty confusing to Python users. This article highlights the differences between the three advanced concepts and guides you on how they may be used effectively.

In Python programming, iterable, iterator and generator are the most confusing concepts because many cannot tell what differentiates them. For example, in the Python 2.7 dictionary data structure, we can use `dict.items()` as well as `dict.iteritems()` for iterating the dictionary. Both the methods work in the same way, but if we talk about memory and performance, there is a huge difference. `dict.items()` returns a list that is iterable and `dict.iteritems()` is a generator.

Containers

Containers are the data structures that hold data or values. They are easy to understand because you can think of them as real-life containers like boxes, trucks, houses, etc. In Python, programming containers are lists, dictionaries and tuples.

Iteration/iterable protocols

- `Iter()` creates an iterator. `Iter()` is called on the iterable to retrieve an iterator.
- `Next()` is called on the iterator to sequentially retrieve elements from iterable.
- When no more elements are available, `next()` will raise `StopIteration`.

Figure1 illustrates the relationship between iterator and iterable.

Iterable

Iterable is an object that implements the `__iter__` method. An iterable is any object (not necessarily a container) that can be a file pointer and return an iterator (with the purpose of returning all of its elements). A Python list object is iterable. Let's check the built-in method of a list.

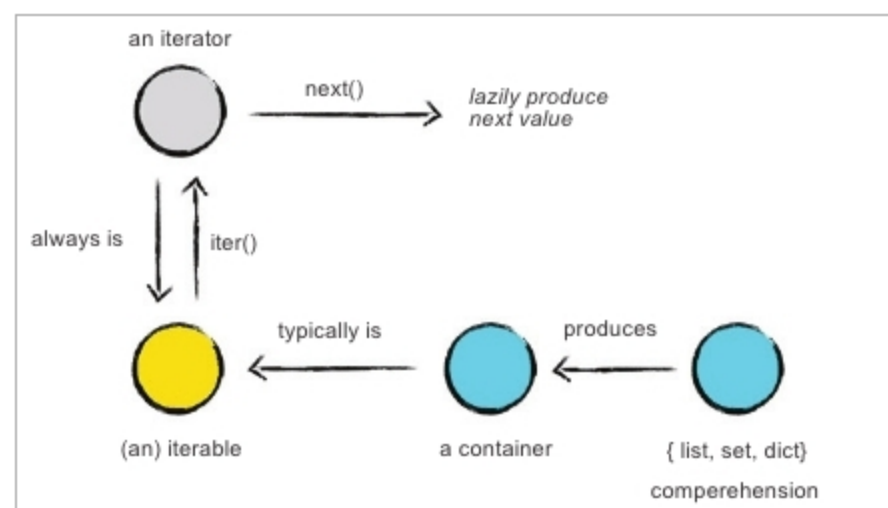


Figure 1: Relation between iterator and iterable

```
>>> dir([])
['_add_', '__class__', '__contains__', '__delattr__',
 '__delitem__', '__dir__', '__doc__', '__eq__', '__format__',
 '__ge__', '__getattribute__', '__getitem__', '__gt__',
 '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__',
 '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__',
 '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__',
 '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__',
 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop',
 'remove', 'reverse', 'sort']
```

The `__iter__` method is shown above. When we can do `iter()` then the `iter()` function calls the magic `__iter__` method.

In order to receive the element one by one, and sequentially, the iterable is converted to an iterator first, and then the `next()` function is used to get the elements from the iterator. Internally, the `next()` function calls the magic method `__next__`.

Let us use the next function with `list` as an argument:

```
>>> list1 = [1,2,3]
>>>
>>> next(list1)
```

Traceback (most recent call last):

```
File "<stdin>", line 1, in <module>
TypeError: 'list' object is not an iterator
>>>
```

You can see the error list is not an iterator. In order to use the `next()` function, make the list iterable. Let's examine the following command:

```
>>> dir(list_new)
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattribute__', '__gt__',
 '__hash__', '__init__', '__init_subclass__', '__iter__',
 '__le__', '__length_hint__', '__lt__', '__ne__', '__new__',
 '__next__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__setstate__', '__sizeof__', '__str__', '__subclasshook__']
>>>
```